EXERCISE 02: MATRIX OPERATIONS — COMPREHENSIVE LAB STUDY GUIDE

> **College Data Science Laboratory Course — Practical Examination Study Guide**
> **Source Code File:** `e02_matrix_operations.py` | **Repository:** `https://github.com/akhilkvu6/ds` | **Domain:** Computational Linear Algebra & Matrix Transformations

1. Program Information and Aim

- **Program Identity:** Program 02 — `e02_matrix_operations.py`.
- **Academic Aim:** To implement a complete interactive linear algebra system that accepts two square matrices of arbitrary dimension $n \times n$ from user terminal input and computes: (1) Dot Product (Matrix Multiplication), (2) Transpose, (3) Trace, (4) Rank, (5) Determinant, (6) Matrix Inverse (with singular matrix validation), and (7) Eigenvalues and Eigenvectors.
- **Required Libraries:** `numpy as np` and `from numpy import linalg`. No external data files required; all inputs are provided interactively via keyboard.

2. Prerequisites and Foundational Concepts

- **Linear Algebra Foundations:** Square matrices, inner products, main diagonal elements, linear independence of row vectors, determinant expansion, adjugate matrices, singularity condition ($\det(A) = 0$), characteristic polynomial, and eigenspace decomposition.
- **Nested List-to-Array Ingestion:** In standard Python, multi-dimensional structures are constructed using nested lists: `matrix = [[e00, e01], [e10, e11]]`. Wrapping this nested list inside `np.array()` creates a contiguous 2D ndarray of shape (n, n) optimized for linear algebra algorithms.
- **Singular Matrix Handling:** If a matrix is singular ($\det(A) = 0$), its inverse does not exist mathematically. Calling `linalg.inv()` directly on a singular matrix triggers a fatal `numpy.linalg.LinAlgError: Singular matrix`. The program handles this via conditional check: `if np.linalg.det(matrix1) != 0: ... else: print("Matrix 1 does not have an inverse.")`.

3. Complete Exam-Ready Algorithm (START ... STOP Format)

ALGORITHM: Interactive Matrix Operations Pipeline

Step 1: START
Step 2: [Import Modules] Import `numpy as np` and import `linalg` from `numpy`.
Step 3: [Define Matrix Ingestion Function `matrixread(n)`]
a. Initialize empty list `matrix = []`.
b. For i from 0 to $n-1$:
i. Initialize empty row list `row = []`.
ii. For j from 0 to $n-1$: Prompt user for integer element at position (i, j) and append to `row`.
iii. Append `row` to `matrix`.
c. Return nested list `matrix`.
Step 4: [Read Dimension and Matrices from User]
a. Prompt user for order of square matrix: `n = int(input(...))`.
b. Call `matrixread(n)`, wrap with `np.array()`, and store in `matrix1`.
c. Call `matrixread(n)`, wrap with `np.array()`, and store in `matrix2`.
d. Print both entered matrices.
Step 5: [Compute Operation 1 — Dot Product] Compute and print matrix product: `np.dot(matrix1, matrix2)`.
Step 6: [Compute Operation 2 — Transpose] Compute and print transpose of Matrix 1: `np.transpose(matrix1)`.
Step 7: [Compute Operation 3 — Trace] Compute and print sum of main diagonal: `np.trace(matrix1)`.
Step 8: [Compute Operation 4 — Rank] Compute and print number of linearly independent rows: `linalg.matrix_rank(matrix1)`.
Step 9: [Compute Operation 5 — Determinant] Compute and print determinant: `np.linalg.det(matrix1)`.
Step 10: [Compute Operation 6 — Inverse]
If `np.linalg.det(matrix1) != 0`: compute and print `linalg.inv(matrix1)`.
Else: print `"Matrix 1 does not have an inverse."`.
Step 11: [Compute Operation 7 — Eigenvalues & Eigenvectors]
Execute `eigenvalues, eigenvectors = linalg.eig(matrix1)`.
Print `eigenvalues` and `eigenvectors`.
Step 12: STOP

4. Complete Theory, Formulas, and Hand-Calculated Worked Example

4.1 Mathematical Formulations of the 7 Operations

1. **Dot Product (Matrix Multiplication):** $C = A \cdot B$ where $C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$.
2. **Transpose (A^T):** Swaps rows and columns: $(A^T)_{ij} = A_{ji}$.
3. **Trace ($\text{tr}(A)$):** Sum of main diagonal elements: $\text{tr}(A) = \sum_{i=1}^n A_{ii}$.
4. **Matrix Rank ($\text{rank}(A)$):** Maximum number of linearly independent row or column vectors, computed via Singular Value Decomposition (SVD).
5. **Determinant ($\det(A)$):** For a 2×2 matrix: $\det(A) = ad - bc$.
6. **Inverse (A^{-1}):** $A^{-1} = \frac{1}{\det(A)} \text{adj}(A)$. Exists if and only if $\det(A) \ne 0$. For a 2×2 matrix: $$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$
7. **Eigenvalues (λ) and Eigenvectors (v):** Satisfy $A v = \lambda v$, or equivalently $\det(A - \lambda I) = 0$.

4.2 Step-by-Step Hand-Calculated Worked Example (2×2)

Sample Input Matrices:
$M_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad M_2 = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

1. **Dot Product:** $$M_1 \cdot M_2 = \begin{bmatrix} (1)(5)+(2)(7) & (1)(6)+(2)(8) \\ (3)(5)+(4)(7) & (3)(6)+(4)(8) \end{bmatrix} = \begin{bmatrix} 5+14 & 6+16 \\ 15+28 & 18+32 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$ 2. **Transpose:** $M_1^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$
3. **Trace:** $\text{tr}(M_1) = 1 + 4 = 5$
4. **Rank:** Row 1 and Row 2 are linearly independent → $\text{rank}(M_1) = 2$
5. **Determinant:** $\det(M_1) = (1)(4) - (2)(3) = 4 - 6 = -2.0$
6. **Inverse:** Since $\det(M_1) = -2 \ne 0$: $$M_1^{-1} = \frac{1}{-2} \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix} = \begin{bmatrix} -2.0 & 1.0 \\ 1.5 & -0.5 \end{bmatrix}$$ 7. **Eigenvalues and Eigenvectors:**
Characteristic equation: $\det(M_1 - \lambda I) = (1-\lambda)(4-\lambda) - 6 = \lambda^2 - 5\lambda - 2 = 0$.
Roots: $\lambda = \frac{5 \pm \sqrt{25 - 4(1)(-2)}}{2} = \frac{5 \pm \sqrt{33}}{2} \approx \frac{5 \pm 5.74456}{2}$
• $\lambda_1 \approx -0.37228, \quad \lambda_2 \approx 5.37228$
• Eigenvectors are normalized columns: $v_1 \approx [-0.82456, \; 0.56577]^T, \; v_2 \approx [-0.41597, \; -0.90938]^T$.

5. Complete Original Source Code and Block Walkthrough

5.1 Verbatim Source Code (`e02_matrix_operations.py`)

```
# Exercise 2: Matrix Operations

# Question 1: Write a NumPy program to create two matrices (read values from user) and find the following:
1. Dot Product 2. Transpose 3. Trace 4. Rank 5. Determinant 6. Inverse 7. Eigen values and eigen vectors.

import numpy as np
from numpy import linalg

def matrixread(n):
    matrix = []

    for i in range(n):
        row = []

        for j in range(n):
            num = int(input("Enter element " + str(i) + "," + str(j) + ": "))
            row.append(num)

        matrix.append(row)

    return matrix


n = int(input("Enter the order of the matrix: "))

print("Enter Matrix 1")
matrix1 = np.array(matrixread(n))

print("Enter Matrix 2")
matrix2 = np.array(matrixread(n))

print("Matrix 1:")
print(matrix1)

print("Matrix 2:")
print(matrix2)

# 1. Dot Product
print("Dot Product:")
print(np.dot(matrix1, matrix2))

# 2. Transpose
print("Transpose of Matrix 1:")
print(np.transpose(matrix1))

# 3. Trace
print("Trace of Matrix 1:")
print(np.trace(matrix1))

# 4. Rank
print("Rank of Matrix 1:")
print(linalg.matrix_rank(matrix1))

# 5. Determinant
print("Determinant of Matrix 1:")
print(np.linalg.det(matrix1))

# 6. Inverse
if np.linalg.det(matrix1) != 0:
```

```
    print("Inverse of Matrix 1:")
    print(linalg.inv(matrix1))
else:
    print("Matrix 1 does not have an inverse.")

# 7. Eigen values and eigen vectors
eigenvalues, eigenvectors = linalg.eig(matrix1)

print("Eigen values:")
print(eigenvalues)

print("Eigen vectors:")
print(eigenvectors)
```

6. Complete Function, Parameter, and Syntax Reference

6.1 Comprehensive 10-Point Technical Breakdown

1. `matrixread(n)` **(Custom Input Function):** Implements double nested `for` loops iterating across rows ($i \in [0, n-1]$) and columns ($j \in [0, n-1]$). Appends integer elements to `row`, then appends `row` to `matrix`, returning a Python 2D nested list.
2. `np.array(object)`**:** Converts Python nested list into contiguous 2D NumPy ndarray. Essential for enabling vectorized linear algebra functions.
3. `np.dot(a, b)`**:** Computes matrix multiplication of a and b. For 2×2 matrices, computes row-by-column inner products. *Mistake:* Do not confuse with element-wise multiplication `a * b`.
4. `np.transpose(a)`**:** Permutes the dimensions of an array, reflecting elements across the main diagonal. For 2D array, swaps axes 0 and 1. Shortcut: `a.T`.
5. `np.trace(a)`**:** Computes the sum of elements along the main diagonal ($\sum A_{ii}$). Returns scalar sum.
6. `linalg.matrix_rank(M, tol=None)`**:** Returns matrix rank using Singular Value Decomposition (SVD). Counts number of singular values greater than the numerical tolerance threshold.
7. `np.linalg.det(a)`**:** Computes the determinant of square matrix a using LU factorization. Output is a 64-bit float.
8. `if np.linalg.det(matrix1) != 0:`**:** Conditional guard preventing fatal division-by-zero crashes when attempting to invert singular matrices.
9. `linalg.inv(a)`**:** Computes multiplicative inverse A^{-1} such that $A \cdot A^{-1} = I$. Requires square and non-singular matrix.
10. `linalg.eig(a)`**:** Computes eigenvalues ($\mathbf{w}$) and right eigenvectors ($\mathbf{v}$). Returns tuple `(eigenvalues, eigenvectors)` where columns of `eigenvectors` correspond to λ_i.

6.2 Technical Coverage Checklist

Technical Concept / Item	Line Number	Status	Exam Relevance
Nested user input function `matrixread()`	Lines 8–20	✓ Verified	Custom matrix ingestion logic
Dynamic dimension parsing `int(input())`	Line 23	✓ Verified	Allows arbitrary $n \times n$ matrices
Nested list to ndarray wrapping	Lines 26, 29	✓ Verified	`np.array(matrixread(n))`
`np.dot(matrix1, matrix2)`	Line 39	✓ Verified	Matrix multiplication syntax
`np.transpose(matrix1)`	Line 43	✓ Verified	Matrix transpose syntax
`np.trace(matrix1)`	Line 47	✓ Verified	Main diagonal summation
`linalg.matrix_rank(matrix1)`	Line 51	✓ Verified	SVD-based rank determination
`np.linalg.det(matrix1)`	Line 55	✓ Verified	Determinant evaluation
Singular matrix conditional check	Lines 58–62	✓ Verified	Error prevention logic for inverse
`linalg.inv(matrix1)`	Line 60	✓ Verified	Multiplicative inverse calculation
`linalg.eig(matrix1)` tuple unpacking	Line 65	✓ Verified	Eigen decomposition syntax

7. Step-by-Step Program Execution Walkthrough

Step	Line	Expression / Action	State & Intermediate Values
1	23	`n = int(input())`	User enters `2`. $n = 2$.
2	26	Read Matrix 1	User inputs `1, 2, 3, 4` → `matrix1 = np.array([[1, 2], [3, 4]])`
3	29	Read Matrix 2	User inputs `5, 6, 7, 8` → `matrix2 = np.array([[5, 6], [7, 8]])`
4	39	`np.dot(matrix1, matrix2)`	Multiplies row by col → `[[19, 22], [43, 50]]`
5	43	`np.transpose(matrix1)`	Swaps rows/cols → `[[1, 3], [2, 4]]`
6	47	`np.trace(matrix1)`	$1 + 4 = 5$
7	51	`linalg.matrix_rank(matrix1)`	2 independent rows → Rank = 2
8	55	`np.linalg.det(matrix1)`	$(1)(4) - (2)(3) = -2.0$
9	58–62	Inverse condition	$\det \ne 0$ is True → Inverts: `[[-2.0, 1.0], [1.5, -0.5]]`
10	65–71	`linalg.eig(matrix1)`	Roots of char poly → $\lambda = [-0.372, 5.372]$, columns v_1, v_2

8. Output and Practical Interpretation

8.1 Sample Terminal Execution Session

```
Enter the order of the matrix: 2
Enter Matrix 1
Enter element 0,0: 1
Enter element 0,1: 2
Enter element 1,0: 3
Enter element 1,1: 4
Enter Matrix 2
Enter element 0,0: 5
Enter element 0,1: 6
Enter element 1,0: 7
Enter element 1,1: 8
Matrix 1:
[[1 2]
 [3 4]]
Matrix 2:
[[5 6]
 [7 8]]
Dot Product:
[[19 22]
 [43 50]]
Transpose of Matrix 1:
[[1 3]
 [2 4]]
Trace of Matrix 1:
5
Rank of Matrix 1:
2
Determinant of Matrix 1:
-2.0000000000000004
Inverse of Matrix 1:
[[-2.   1. ]
 [ 1.5 -0.5]]
Eigen values:
[-0.37228132  5.37228132]
Eigen vectors:
[[-0.82456484 -0.41597356]
 [ 0.56576746 -0.90937671]]
```

9. Exam Preparation: Lab Record Answer and Model Questions

9.1 Lab Record Standard Format (Write in 1 Page)

EXERCISE 2: MATRIX OPERATIONS

AIM: To write a Python program using NumPy to read two matrices from user input and compute Dot Product, Transpose, Trace, Rank, Determinant, Inverse, and Eigenvalues/Eigenvectors.

ALGORITHM:
1. Import `numpy as np` and `from numpy import linalg`.
2. Define function `matrixread(n)` to read $n \times n$ elements via nested loops and return a nested list.
3. Read dimension n, then read `matrix1` and `matrix2` and convert to NumPy arrays.
4. Compute Dot Product using `np.dot(matrix1, matrix2)`.
5. Compute Transpose using `np.transpose(matrix1)`.
6. Compute Trace using `np.trace(matrix1)`.
7. Compute Rank using `linalg.matrix_rank(matrix1)`.
8. Compute Determinant using `np.linalg.det(matrix1)`.
9. Check if determinant is non-zero; if so, compute Inverse using `linalg.inv(matrix1)`, else print non-invertible message.
10. Compute Eigenvalues and Eigenvectors using `linalg.eig(matrix1)` and display.

RESULT: All seven fundamental matrix operations were successfully performed on user-input matrices.

9.2 Common Exam Questions and Model Answers

- **2-Mark Question:** What is a singular matrix and why does it lack an inverse?
  Answer: A singular matrix is a square matrix whose determinant is zero ($\det(A) = 0$). Since the inverse formula involves dividing by the determinant ($A^{-1} = \frac{1}{\det(A)}\text{adj}(A)$), division by zero is undefined, and thus no inverse exists.
- **3-Mark Question:** What is the difference between `np.dot(A, B)` and `A * B`?
  Answer: `np.dot(A, B)` performs standard mathematical matrix multiplication (row-by-column inner products). In contrast, `A * B` performs element-wise multiplication ($A_{ij} \times B_{ij}$) where matrices must have identical shapes.
- **5-Mark Question:** Explain eigenvalues and eigenvectors, and interpret the structure of the output returned by `linalg.eig()`.
  Answer: For square matrix A, a non-zero vector v is an eigenvector and scalar λ is its eigenvalue if $Av = \lambda v$. `linalg.eig(A)` returns a tuple `(w, v)`. The 1D array `w` contains the eigenvalues. The 2D array `v` contains the corresponding eigenvectors stored as *column vectors*, such that column `v[:, i]` corresponds to eigenvalue `w[i]`.

9.3 Predict-the-Output Questions

1. **Question:** What is `np.trace(np.array([[2, 5], [1, 7]]))`?
   Answer: $2 + 7 = 9$.
2. **Question:** What is `linalg.matrix_rank(np.array([[1, 2], [2, 4]]))`?
   Answer: `1` (since Row 2 is an exact scalar multiple of Row 1, only 1 linearly independent row exists).

9.4 Identify-the-Function Questions

1. **Question:** Which function in `numpy.linalg` computes the eigenvalues and eigenvectors of a square matrix?
   Answer: `linalg.eig()`.
2. **Question:** Which function calculates the matrix rank using singular value decomposition?
   Answer: `linalg.matrix_rank()`.
3. **Question:** Which function computes the multiplicative inverse of a matrix?
   Answer: `linalg.inv()`.

10. Comprehensive Viva Voce Questions and Answers (16 Exam Questions)

Q1: What condition must be met for two matrices A and B to be multiplied using `np.dot(A, B)`?
Answer: The number of columns in matrix A must equal the number of rows in matrix B (inner dimensions must match: $(m \times k) \cdot (k \times n) \to (m \times n)$).

Q2: What is the trace of a matrix?
Answer: The trace is the sum of the elements lying on the main diagonal of a square matrix ($\text{tr}(A) = \sum A_{ii}$).

Q3: How does `linalg.matrix_rank()` compute matrix rank?
Answer: It uses Singular Value Decomposition (SVD) and counts the number of singular values that exceed a numerical floating-point tolerance threshold.

Q4: Why does the program check `if np.linalg.det(matrix1) != 0:` before inverting?
Answer: If $\det(A) = 0$, the matrix is singular and has no inverse. Calling `linalg.inv()` on it raises a fatal `LinAlgError` runtime exception. The conditional check prevents a program crash.

Q5: What are eigenvalues and eigenvectors?
Answer: Vectors v whose direction remains unchanged when linear transformation A is applied, only being scaled by factor λ: $Av = \lambda v$.

Q6: In the output of `linalg.eig(A)`, are eigenvectors stored as rows or columns?
Answer: They are stored as **columns**. Column i (accessed via `eigenvectors[:, i]`) corresponds to the eigenvalue at index i in `eigenvalues[i]`.

Q7: Are eigenvectors returned by `linalg.eig()` normalized?
Answer: Yes, scikit-learn and NumPy normalize each eigenvector to have unit Euclidean norm ($\|v\|_2 = 1$).

Q8: What is the transpose of a symmetric matrix?
Answer: The matrix itself: $A^T = A$.

Q9: What is the determinant of an Identity matrix?
Answer: $\det(I) = 1.0$.

Q10: What does a matrix rank of n for an $n \times n$ matrix signify?
Answer: It signifies that the matrix has "full rank", all n rows and columns are linearly independent, the determinant is non-zero, and the matrix is invertible.

Q11: What is the result of multiplying a matrix by its inverse: $A \cdot A^{-1}$?
Answer: The Identity matrix I, which has ones along the main diagonal and zeros elsewhere.

Q12: Why does the determinant calculation print `-2.0000000000000004` instead of exactly `-2.0`?
Answer: Due to IEEE 754 standard 64-bit floating-point rounding representation in computer hardware.

Q13: What is the purpose of the nested loop in `matrixread()`?
Answer: The outer loop iterates through rows, and the inner loop iterates through columns, reading each individual element from user input to construct a 2D grid.

Q14: Can `np.trace()` be computed on non-square matrices?
Answer: Yes, NumPy computes the sum of the elements along the main diagonal A_{ii} even for rectangular matrices where row and column indices match.

Q15: What is the time complexity of multiplying two $n \times n$ matrices using standard algorithms?
Answer: $\mathcal{O}(n^3)$ for standard matrix multiplication, though optimized BLAS implementations can achieve lower sub-cubic complexities.

Q16: How does linear algebra connect to machine learning models like linear regression?
Answer: Linear regression computes parameter weights using the closed-form Normal Equation: $\boldsymbol{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$, which relies directly on transpose, dot product, and matrix inverse.

11. Common Mistakes, Pitfalls, and Debugging Guide

Common Error / Pitfall	Root Cause	Correction / Debugging Technique
`LinAlgError: Singular matrix`	Calling `linalg.inv()` on a matrix with zero determinant.	Always guard inversion with `if np.linalg.det(m) != 0:`.
Using `*` instead of `np.dot()`	Writing `matrix1 * matrix2` performs element-wise multiplication.	Use `np.dot(matrix1, matrix2)` for mathematical matrix multiplication.
Extracting eigenvectors as rows	Assuming `eigenvectors[0]` is the first eigenvector.	Eigenvectors are columns! Extract via `eigenvectors[:, 0]`.
Forgetting `int()` on user input	Reading elements as strings, causing string concatenation during operations.	Wrap `input()` with `int(...)` or `float(...)`.
Non-matching matrix shapes in dot product	Attempting dot product where columns of A $\ne$ rows of B.	Ensure inner dimensions match: $(m \times k)$ and $(k \times n)$.

12. Memory Tricks, Mnemonics, and Last-Minute Rapid Revision

12.1 Mnemonics & Memory Aids

- **Mnemonic for Matrix Operations — "D-T-T-R-D-I-E":**
  Dot product • **T**ranspose • **T**race • **R**ank • **D**eterminant • **I**nverse • **E**igenvalues.
- **Determinant Rule:** "Main minus Anti" — $ad - bc$.
- **Inverse Rule:** "Swap main diagonal, negate anti-diagonal, divide by determinant".

12.2 Top 5 High-Priority Viva Questions (Last-Minute Refresh)

1. What is the difference between `np.dot(A, B)` and `A * B`?
Answer: `np.dot` performs mathematical matrix multiplication (row × col); `A * B` performs element-wise multiplication.

2. Why must we check `det != 0` before computing `linalg.inv()`?
Answer: A matrix with determinant 0 is singular and has no inverse; inverting it crashes with `LinAlgError`.

3. How are eigenvectors arranged in the output of `linalg.eig()`?
Answer: As **columns** of the 2D returned array, not rows.

4. What is the trace of a matrix?
Answer: The sum of the numbers along the main diagonal ($\sum A_{ii}$).

5. How is matrix rank defined?
Answer: The maximum number of linearly independent row or column vectors in the matrix.

13. Cross-Program Understanding (Data Science Pipeline Progression)

Linear algebra operations form the mathematical foundation for machine learning algorithms:

Linear Algebra Concept	NumPy Function	Direct Use in ML Curriculum
Dot Product	`np.dot()`	Multiple Linear Regression ($y = \mathbf{X}\boldsymbol{\beta}$) and Neural Network forward passes
Matrix Transpose	`np.transpose()`	Normal Equation feature matrix transposition ($\mathbf{X}^T$) in Exercise 10
Matrix Inverse	`linalg.inv()`	Solving Ordinary Least Squares weights: $\boldsymbol{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
Euclidean Norm / Distance	Derived from Dot Product	Distance calculations in KNN (Exercise 05) and K-Means (Exercise 09)
Eigenvalues / Vectors	`linalg.eig()`	Principal Component Analysis (PCA) dimensionality reduction and spectral clustering